

问题A.方形王国

在方形王国中，编号为 1 至 n 的 n 居民独自居住在一根高高的石柱顶上。第 i 位居民的柱子距地面高度为 $(i + \frac{b}{a})^2$ 个单位。

由于每个人都住得很高，拜访邻居的唯一方法就是使用梯子。王国为每一对居民建造一个梯子。每个梯子的长度恰好是它所连接的两根柱子之间高度的绝对差。

总共有 $\frac{n(n-1)}{2}$ 个梯子，管理它们变得很困难。系统会询问您第 k 个梯子的长度（按长度升序排列）。这些信息将帮助王国计划维修、交付和社区活动。

输入

唯一的一行包含四个整数 n ($2 \leq n \leq 10^{12}$)、 k ($1 \leq k \leq \min \left\{ \frac{n(n-1)}{2}, 10^{12} \right\}$)、 a ($1 \leq a \leq 10^6$) 和 b ($0 \leq b \leq 10^{12}$)。

输出

输出一行包含两个整数 p 和 q ，表示第 k 个梯子的长度按长度升序可以表示为 $\frac{p}{q}$ ，有两个整数 p 和 q ($p \geq 0$ 、 $q \geq 1$ 、 $\gcd(p, q) = 1$)，其中 $\gcd(p, q)$ 是 p 的最大公约数， q_0 。

例子

standard input
3 1 3 1
standard output
11 3
standard input
3 2 3 1
standard output
17 3
standard input
3 3 3 1
standard output
28 3
standard input
1414215 1000000000000 1000000 1000000000000
standard output
4823373069559 1

此页故意留空

问题 B. Buggy 绘画软件 I

耳廓狐 Sulfox 是一位喜欢在业余时间创作数字艺术的程序员。经过一天的调试，疲惫不堪的他像往常一样打开绘画软件，却发现软件本身在最新的更新后也漏洞百出，其中一个主要问题是每次编辑像素时都会出现明显的卡顿。

绘画软件将画布视为有序的图层堆栈。每层都是具有 n 行和 m 列的像素网格，每个像素在 $\{0, 1, \dots, nm\}$ 中保存可见值，其中 0 表示透明，正值是 nm 不同颜色的标识符。由于图层在任意位置都可能相互遮挡，因此合成图像的可见值等于所有图层中最上面的非透明像素的可见值；如果该位置的所有图层都是透明的，则可见值为 0。

Sulfox 已经确切地知道他想要绘制什么：由可见值矩阵 $(p_{i,j})_{n \times m}$ 表示的目标图像。为了尽快完成，他希望最大限度地减少软件错误造成的总“滞后成本”。从无层开始，Sulfox 可以执行以下操作任意多次：

- 创建图层：在顶部免费插入一个新图层并将其初始化为恒定颜色图层
将每个像素设置为相同的所选颜色 c ($1 \leq c \leq nm$)，或将每个像素设置为空层透明的；r
l
- 画笔工具：花费 a 将任意图层的任意单个像素设置为任意选定的颜色 c ($1 \leq c \leq nm$)；
- 橡皮擦工具：花费 b 使任何图层的任何单个像素透明。

帮助 Sulfox 确定执行操作所需的最低总成本，以便复合图像与目标图像匹配，即两幅图像对应位置的可见光值相等。e

输入

输入的第一行包含一个整数 T ($1 \leq T \leq 10^5$)，表示测试用例的数量。对于每个测试用例：

第一行包含四个整数 n 、 m ($1 \leq n, m \leq 500$)、 a 和 b ($1 \leq a, b \leq 10^6$)，分别表示图像尺寸、使用画笔工具的成本和使用橡皮擦工具的成本。

接下来的 n 行的第 i 行包含 m 整数 $p_{i,1}, p_{i,2}, \dots, p_{i,m}$ ($0 \leq p_{i,j} \leq nm$)，其中 $p_{i,j}$ 表示目标图像在第 i 行和第 j 列的可见值。

保证所有测试用例的 nm 之和不超过 10^6 。

输出

对于每个测试用例，输出一行包含一个整数，表示最小总成本。

例子

standard input	standard output
3 1 2 3 2 0 1 2 2 1 1 1 0 2 3 3 3 5 3 2 4 4 4 1 4 4 4 2	2 3 11

笔记



对于示例案例的第一个测试用例，Sulfox可以先创建一个颜色为1的图层，然后花费 $b = 2$ 使第一行第一列的像素透明。

问题C. Buggy绘画软件II

这是一个沟通问题。

好吧……在问题 B. Buggy 绘画软件 I 中的噩梦般的口吃中幸存下来后，Sulfox 很快就面临着一个更严重的错误——他的绘画软件以黑白格式保存任何彩色图像。

这里，彩色图像支持标记为 1 到 m 的 m 不同颜色，并且可以将其视为由颜色标签组成的长度为 $3m$ 的序列，其中每种颜色恰好出现 3 次。黑白图像仅支持黑色（标记为 0）和白色（标记为 1），并且可以被视作二进制序列。保存彩色图像时，软件选择 $S = \{1, 2, \dots, m\}$ 的非空二分 (S_0, S_1) ，因此 $S_0 \cup S_1 = S$ ， $S_0 \cap S_1 = \emptyset$ ，并且 S_0 和 S_1 都是非空，并将原始序列转换为相同长度的二进制序列，将 S_0 中的每种颜色映射为 0，将 S_1 中的每种颜色映射为 1。生成的二进制序列将是保存的黑白图像。

为了智取这个错误，Sulfox 希望创建具有相应整数特征值的 n 彩色图像， $x_1, x_2, \dots, x_n$ （值可以重复），范围从 1 到 m 。您的任务是帮助 Sulfox 设计这些彩色图像，以便即使在颜色丢失后，每个图像的特征仍然可识别。更具体地，第 i 个彩色图像 ($1 \leq i \leq n$) 必须满足：无论如何选择非空二分 (S_0, S_1) ，其特征值 x_i 总是可以从保存的黑白图像中恢复。

为了验证您的设计的正确性，您的程序将在每个测试中运行两次：首先构建 n 彩色图像，然后从 n 黑白图像恢复原始特征值。第二次运行将准确接收由独立选择的非空二分区在第一次运行中输出的每个 n 序列映射的 n 二进制序列。除此之外，第二次运行不会从第一次运行中收到任何附加信息。如果您正确地从每个黑白图像中恢复相应的特征值，您的解决方案就通过了测试。

请注意，所应用的二分法是不可预测的，并且可能会适应您的设计，并且不同的二分法可能适用于具有相同特征值的彩色图像。

输入

第一行包含三个整数 op ($op \in \{1, 2\}$)、 n 和 m ($2 \leq n, m \leq 5 \times 10^5$ 、 $nm \leq 10^6$)，分别表示程序运行的次数、彩色图像的数量和不同颜色的数量。保证 n 和 m 在两次运行中保持一致。

如果 $op = 1$ ，您的程序必须构造彩色图像。接下来的 n 行，其中第 i 行包含单个整数 x_i ($1 \leq x_i \leq m$)，表示要设计的第 i 图像的特征值。

如果 $op = 2$ ，您的程序必须恢复特征值。接下来的 n 行，每行描述一个黑白图像。每行包含 $3m$ 个空格分隔的 0 或 1 整数，形成一个由您设计的彩色图像之一生成的二进制序列。这些 n 序列以任意顺序给出，并且不一定对应于第一次运行中给出特征值的顺序。

输出

如果 $op = 1$ ，则输出 n 行，其中第 i 行描述了输入中第 i 个特征值 x_i 的设计彩色图像。每行包含 $3m$ 个空格分隔的整数。每行上的序列必须是有效的彩色图像，即从 1 到 m 的每个整数恰好出现 3 次的序列。

如果 $op = 2$ ，则输出 n 行。对于输入中给出的每个 n 二进制序列，输出一行包含一个整数，表示从该图像恢复的原始特征值。输出的顺序必须与输入中二进制序列的顺序相对应。

例子

standard input	standard output
1 2 3 1 2	1 1 1 2 2 2 3 3 3 1 2 3 1 2 3 1 2 3
2 2 3 1 1 0 1 1 0 1 1 0 0 0 0 1 1 1 0 0 0	2 1

笔记

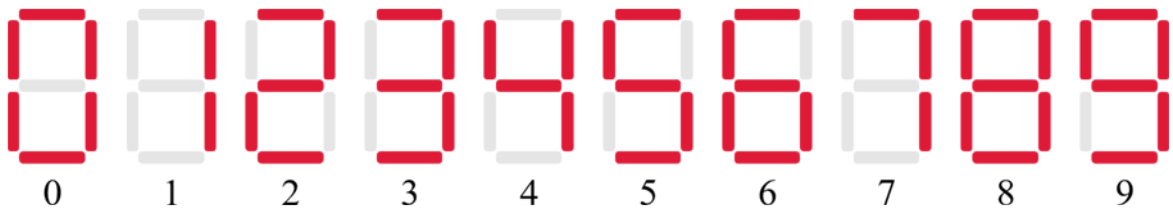
该示例显示了某个解决方案在示例案例上的两次运行。

在第一次运行中，使用了一个简单的（对于一般情况来说是不正确的）策略：特征值 1 被设计为三种相同颜色的连续块，特征值 2 被设计为三种不同颜色的重复图案。

在第二次运行中，第一个二进制序列（来自二分区 ($S_0 = \{3\}, S_1 = \{1, 2\}$)）通过其重复模式 3 来识别，以恢复特征值 2，而第二个二进制序列（来自二分区 ($S_0 = \{1, 3\}, S_1 = \{2\}$)）通过其连续的 3 块来识别，以恢复特征值 1。由于法官以交换顺序提供这些序列，因此输出为2 然后 1。

问题D. LED显示屏改造

您有一个能够显示 n 位整数的 LED 显示屏，其中每个数字都使用 `th` 表示标准 7 段布局。



最初，每个数字块中的每个 7 位数字段可能处于以下三种状态之一：

- 功能性（用 “w” 表示）：行为正确，即根据显示的数字的要求点亮或关闭；
- Stuck ON（用 “1” 表示）：无论显示数字如何，始终亮起；
- 卡在关闭状态（用 “0” 表示）：无论显示数字如何，始终不亮。

您最多可以更新 k 个数字段，即最多将 k 个数字段转换为功能。您需要最大化能够正确显示的整数数量，并找到达到最大值的方法。请注意，您可以更新初始可用的数字段，并且每个数字段最多可以更新一次。如果至少有一个数字段以一种方式翻新但没有以另一种方式翻新，则两种翻新方式被认为是不同的。

当且仅当满足以下条件时，整数才会正确显示：

- 它没有前导零，除非整数恰好为 0；
- 它在显示屏上右对齐：如果整数有 d ($1 \leq d \leq n$) 位，则仅使用最右边的 d 数字块，最左边的 ($n - d$) 块应留空且不显示任何内容；
- 对于每个 d 使用的数字块，当功能段显示该数字的正确状态并且故障段仍停留在其状态时，所有 7 个数字段都与目标数字的显示模式匹配。

输入

输入的第一行包含一个整数 T ($1 \leq T \leq 100$)，表示测试用例的数量。对于每个测试用例：

第一行包含两个整数 n ($1 \leq n \leq 9$) 和 k ($0 < k < 7n$)，分别表示 LED 显示屏可以显示的位数和可以修复的位数。

接下来的 7 行每行包含一个恰好 $(5n - 1)$ 个字符的字符串，以 ASCII 图像的形式描述 LED 显示的初始状态。

显示由 n 数字块组成，每个数字块占据 4 列，由单列间隙分隔。所有数字段均由两个相同的字符 “w”、“0” 或 “1” 表示，所有其他位置均用字符 “.” 填充，纯粹用于格式化。有关详细信息，请参阅示例输入和注释。

输出

对于每个测试用例，输出一行包含两个整数，表示通过翻新最多 k 个数字段可以正确显示的最大整数个数，以及达到最大值的方法数。

例子

standard input	
2	
3 2	
.00...00...00.	
0..0.0..0.0..1	
0..0.0..0.0..1	
.00...00...00.	
0..W.0..0.0..1	
0..W.0..0.0..1	
.00...00...00.	
9 62	
.WW...WW...WW...WW...WW...WW...WW...WW...WW.	
W..W.W..W.W..W.W..W.W..W.W..W.W..W.W..W.W..W	
W..W.W..W.W..W.W..W.W..W.W..W.W..W.W..W.W..W	
.WW...WW...WW...WW...WW...WW...WW...WW...WW.	
W..W.W..W.W..W.W..W.W..W.W..W.W..W.W..W.W..W	
W..W.W..W.W..W.W..W.W..W.W..W.W..W.W..W.W..W	
.WW...WW...WW...WW...WW...WW...WW...WW...WW.	
standard output	
2 23	
1000000000 9223372036854775807	

笔记

LED 以 ASCII 图像形式显示的每个数字块中，数字段如下：

S:

Segment	Part	Positions (row, column)
0	Top horizontal	(1, 2), (1, 3)
1	Upper-left vertical	(2, 1), (3, 1)
2	Upper-right vertical	(2, 4), (3, 4)
3	Middle horizontal	(4, 2), (4, 3)
4	Lower-left vertical	(5, 1), (6, 1)
5	Lower-right vertical	(5, 4), (6, 4)
6	Bottom horizontal	(7, 2), (7, 3)

数字 0, 1, 2, ..., 9 (1 = 点亮, 0 = 熄灭) 的显示模式如下：

Digit	Pattern (segments 0, 1, 2, ..., 6)	Explanation
0	1110111	All except middle
1	0010010	Upper-right and lower-right
2	1011101	Top, upper-right, middle, lower-left, and bottom
3	1011011	Top, upper-right, middle, lower-right, and bottom
4	0111010	Upper-left, upper-right, middle, and lower-right
5	1101011	Top, upper-left, middle, lower-right, and bottom
6	1101111	All except upper-right
7	1010010	Top, upper-right, and lower-right
8	1111111	All segments
9	1111011	All except lower-left

问题 E. 见机行事

您正在玩一种流行的纸牌游戏。有 $2n$ 张不同的牌，标记为 1 到 $2n$ ，随机均匀洗牌并堆积成一副牌。你抽取最上面的 n 卡作为你的起手牌，剩余的牌组顺序对你来说是隐藏的。每回合，你从手上选择任意一张牌，将其放在牌堆底部，然后抽出当前的顶牌，这样你的手牌大小仍为 n 。

您有 m 个任务需要按顺序完成：第 i 个任务要求您打出标记为 a_i 的卡牌，并且只有在第 $(i - 1)$ 个任务完成后才会激活（除非它是第一个任务，该任务最初是激活的）。在第 i 个任务激活之前打出标记为 a_i 的牌不会计入第 i 个任务。您知道任何 $a_1, a_2, \dots, a_m$ (值都可能会提前重复)。

假设您在最佳纯策略下打牌，计算在随机初始洗牌中完成所有 m 任务所需的最小预期回合数，模 998 24 4 353。

形式上，纯策略意味着在每个回合中，选择的牌只取决于可观察的历史，并且相同的已知信息（初始手牌、过去回合中各自打出和抽出的牌以及任务完成状态）确定性地产生相同的选择牌。对于纯策略 σ 和初始洗牌 π ，令 $f_\sigma(\pi)$ 为跟随 σ 时完成所有 m 任务所需的回合数；要计算的值等于 $(\min_\sigma E[f_\sigma(\pi)]) \bmod 998\,244\,353$ ，其中期望 $E[\cdot]$ 接受所有 $(2n)$ 上的均匀分布！初始牌组可能会洗牌。

输入

输入的第一行包含一个整数 T ($1 \leq T \leq 1\,000$)，表示测试用例的数量。对于每个测试用例：

第一行包含 两个整数 n 和 m ($2 \leq n \leq 5\,000, 1 \leq m \leq 2 \times 10^5$)。

第二行包含 m 整数 $a_1, a_2, \dots, a_m$ ($1 \leq a_i \leq 2n$)。

保证所有测试用例的 n 之和不超过 5 000，所有测试用例的 m 之和不超过 2×10^5 。

输出

对于每个测试用例，由于最小预期匝数可以用不可约分数 $\frac{p}{q}$ 表示，因此只需在一行中打印满足 $0 \leq r < 998\,244\,353$ 和 $r \cdot q \equiv p \pmod{998\,244\,353}$ 的整数 r 。可以证明这样的 r 存在并且是唯一的。

例子

standard input	standard output
3	249561090
2 1	299473317
1	748684517
3 7	
2 5 1 2 6 2 1	
1000 2	
1116 1116	

笔记

对于示例案例的第一个测试用例，一个最优的纯策略是：如果标有 1 的牌在你的初始手牌中，则立即打出；否则重复打出任何一张牌，直到你抽到它并在下一个回合打出它。因此，最小预期匝数为 $\frac{1}{2} \times 1 + \frac{1}{4} \times 2 + \frac{1}{4} \times 3 = \frac{7}{4}$ 。

此页故意留空

问题 F. 超越时间的纽带

耳廓狐萨福克斯建造了一个迷宫，并邀请他的两个朋友爱丽丝和鲍勃来尝试一下。迷宫可以抽象为一个简单的、连通的无向图，具有 n 顶点和 m 边，其中 Alice 和 Bob 从两个不同的指定顶点开始。

在任何玩家移动之前，Sulfox 必须为每个无向边选择一个方向，将迷宫变成有向图。然后，为了找到对方，两名玩家在定向迷宫内轮流移动。每轮他们同时独立行动，不留痕迹，也不交流；相反，他们采取了“最无脑”的策略：

- 如果玩家当前位于具有一条或多条出边的顶点，则他们任意遍历出边到相邻顶点；
- 否则，如果它们当前的顶点没有出边，它们将保留在该顶点。

萨福克斯对自己的迷宫设计技巧感到自豪，他想调整通道的方向，让爱丽丝和鲍勃永远不会相遇。给定无向图和 Alice 和 Bob 的起始顶点，请帮助 Sulfox 确定每条边的方向，以便无论 Alice 和 Bob 在每一轮中如何做出选择，他们在任何一轮结束时都不会占据相同的顶点。如果此类方向不存在，请报告。

输入

输入的第一行包含一个整数 T ($1 \leq T \leq 1\,000$)，表示测试用例的数量。对于每个测试用例：

第一行包含四个整数 n ($2 \leq n \leq 300$)、 m ($n-1 \leq m \leq \frac{n(n-1)}{2}$)、 x 和 y ($1 \leq x, y \leq n$ 、 $x \neq y$)，分别表示迷宫中的顶点数和边数，以及 Alice 和 Bob 的起始顶点。

接下来的每行 m 包含两个整数 u 和 v ($1 \leq u, v \leq n$)，描述连接顶点 u 和 v 的无向边。保证给定的图是连通的并且不包含自环或重边。

是保证 d 最多有 1 个测试用例 n 大于 th 30。

输出

对于每个测试用例：

如果可以调整所有边的方向，以便 Alice 和 Bob 在任何一轮结束时都不会占据相同的顶点，请在第一行打印 “Yes”（不带引号）。

然后输出 m 行，每行包含两个整数 u' 和 v' ($1 \leq u', v' \leq n$)，描述从 u' 到 v' 的有向边。无序对 (u', v') 必须对应于输入中现有的无向边。边缘可以按任何顺序打印。

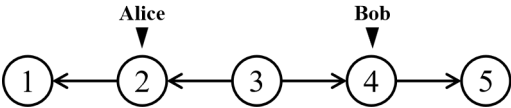
如果不存在有效方向，则输出包含 “No”（不带引号）的单行。

例子

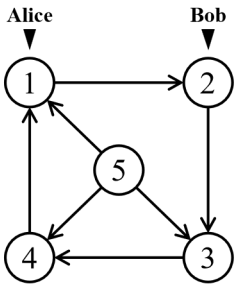
standard input	standard output
3	Yes
5 4 2 4	2 1
1 2	3 2
2 3	3 4
3 4	4 5
4 5	Yes
5 7 1 2	1 2
1 2	2 3
2 3	3 4
3 4	4 1
4 5	5 1
1 5	5 3
3 5	5 4
1 4	No
2 1 1 2	
1 2	

笔记

对于示例案例的第一个测试案例，有效方向说明如下。在第一轮中，Alice 必须从顶点 2 移动到顶点 1，而 Bob 必须从顶点 4 移动到顶点 5。在这一轮之后，两个玩家都处于没有出边的顶点，并且将永远留在那里，因此永远不会相遇。



对于示例案例的第二个测试案例，有效方向说明如下。两个玩家都被限制在循环 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$ 中。由于他们在每一轮中都沿着循环前进一步，所以 Alice 将始终落后 Bob 一步，因此永远不会相遇。



对于示例案例的第三个测试用例，只有一条边连接两个顶点，指示它 $1 \rightarrow 2$ 强制 Alice 移动到 Bob 的顶点，而指示它 $2 \rightarrow 1$ 强制 Bob 移动到 Alice 的顶点。由于一名玩家总是加入固定玩家的行列，因此无论方向如何，相遇都是不可避免的。

问题G. 碰撞损坏

在基于物理的 2D 视频游戏中，两个角色 P 和 Q 在战场上移动。每个角色都由一个凸多边形表示，该多边形充当其碰撞盒。当两个碰撞框重叠时，游戏计算它们相交的面积作为碰撞伤害。

然而，由于竞技场中不可预测的风流，角色 Q 可以被任意平移向量 $\mathbf{t} = (t_x, t_y)$ 位移，但它不能旋转或翻转。您需要计算当 Q 均匀且随机放置以使其实际与 P 碰撞时的预期碰撞损坏，即它们的交集具有正面积。

更正式地，将 $f(\mathbf{t})$ 定义为表示字符 P 的多边形与表示字符 Q 的多边形沿 $\mathbf{t}$ 平移后的交集面积。令 $D \subseteq \mathbb{R}^2$ 为 $f(\mathbf{t}) > 0$ 的所有平移向量 $\mathbf{t}$ 的集合。可以证明 D 具有正面积，用 $|D|$ 表示。您需要计算 $\frac{1}{|D|} \iint_D f(\mathbf{t}) d\mathbf{t}$ 。

输入

输入的第一行包含一个整数 T ($1 \leq T \leq 300$)，表示测试用例的数量。对于每个测试用例：

- 第一行包含两个整数 n 和 m ($3 \leq n, m \leq 1\,000$)，分别表示两个凸多边形的顶点数。
- 然后是 n 行，每行包含两个整数 x 和 y ($-10^4 \leq x, y \leq 10^4$)，给出表示字符 P 的多边形的顶点坐标 (x, y) 。
- 然后是 m 行，每行包含两个整数 x 和 y ($-10^4 \leq x, y \leq 10^4$)，给出表示字符 Q 的多边形的顶点坐标 (x, y) 。
- 保证凸多边形的顶点按逆时针顺序给出，并且没有三个顶点共线。

保证所有测试用例的 n 之和和 m 之和不超过 1000。

输出

对于每个测试用例，输出一行包含实数，表示预期的碰撞损坏。如果您的答案的绝对或相对误差不超过 10^{-6} ，则可以接受。正式来说，假设你的输出是 a ，陪审团的答案是 b ，并且当且仅当 $\frac{|a-b|}{\max(1, |b|)} \leq 10^{-6}$ 你的输出被接受。

例子

standard input	standard output
2	0.083333333333
3 3	0.125000000000
0 0	
1 0	
0 1	
0 0	
1 0	
0 1	
3 3	
0 0	
1 0	
0 1	
0 1	
1 0	
1 1	

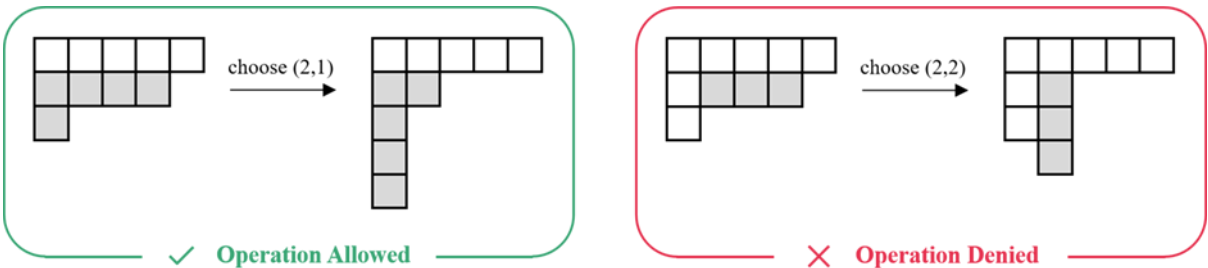
问题 H. 可爱的年轻图计数

杨氏图是一组有限的单元格，排列成左对齐的行，行长度按非递增顺序排列。它可以通过满足 $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_r \geq 1$ 的无序整数分区 $\lambda = (\lambda_1, \lambda_2, \dots, \lambda_r)$ 来唯一表示，其中 r 对应于 Young 图的行数，而 λ_i 对应于 $i = 1, 2, \dots, r$ 的第 i 行中的单元格数。

杨氏图 λ 的共轭是通过转置行和列获得的另一个杨氏图，由共轭分区 λ^T 表示。更具体地说，如果 λ 具有 $c = \lambda_1$ 列，则 $\lambda^T = (\mu_1, \mu_2, \dots, \mu_c)$ where $\mu_j = |\{i : \lambda_i \geq j\}|$ for $j = 1, 2, \dots, c$ 。

对于属于 λ 的第 i 行和第 j 列的单元格，令锚定于 (i, j) 的子图为 λ 与 $x \geq i$ 和 $y \geq j$ 的所有单元格 (x, y) 的集合。很容易看出，当以 (i, j) 为左上角时，该集合本身就是杨氏图。

将杨图 λ 的可爱度定义为可以通过执行任意次数（可能为零）次的以下操作从 λ 获得的不同杨图的数量：选择当前杨图的任何单元格 (i, j) ，获取锚定在 (i, j) 的子图，并将其替换为锚定在同一位置的共轭图。当且仅当整个单元集仍形成有效的杨图时才允许该操作；否则操作将被拒绝并撤消，如下所示。



给定一个非递增的正整数序列 $a_1, a_2, \dots, a_{n_0}$ 。对于每个 $i = 1, 2, \dots, n$ ，计算杨图 $\lambda^{(i)}$ 的可爱度，模 998 244 353，其中 $\lambda^{(i)} = (a_1, a_2, \dots, a_i)$ 。

输入

第一行包含一个整数 n ($1 \leq n \leq 10^6$)，表示给定序列的长度。
第二行包含 n 整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$)。保证是 $a_1 \geq a_2 \geq \dots \geq a_{n_0}$

输出

输出以空格分隔的 n 个整数，其中第 i 个整数表示 Young 图 $\lambda^{(i)}$ 的可爱度，模 998 244 353。

例子

standard input	standard output
3 3 2 1	2 3 1

笔记

对于示例案例：

- 从 $\lambda^{(1)}$ 可以得到不同的杨氏图是 (3) 和 (1, 1, 1)；
- 从 $\lambda^{(2)}$ 可以获得的不同杨氏图是 (3, 2)、(3, 1, 1) 和 (2, 2, 1)；
- 可以从 $\lambda^{(3)}$ 获得的唯一杨图是 (3, 2, 1) 本身。

此页故意留空

问题一. 志愿者模拟器

ICPC竞赛存在问题多、竞赛周期长等问题。总共有 n 个已接受的提交，每个提交都代表团队对某个问题的成功尝试。当一个团队针对某个问题提交了已接受的提交时，我们就说该团队已经解决了该问题。请注意，如果一个团队针对同一问题提交了多个已接受的解决方案，则只有第一个提交的解决方案被视为首次解决该问题。

您需要根据以下规则确定每次提交是否应奖励气球：

- 计分板冻结前（提交时间 < 240分钟）：当团队第一次解决问题时，奖励与该问题对应的气球。
- 记分板冻结后（提交时间 $\geq$ 240分钟）：当团队第一次解决问题时，当且仅当该团队迄今为止收到的气球总数严格少于3个时，才奖励与该问题相对应的气球。

输入

第一行输入包含 n ($1 \leq n \leq 5\,000$)，表示接受的提交数量。
接下来的 n 行按时间顺序描述提交内容。每次提交包含三个整数 a ($1 \leq a \leq 410$)、 b ($1 \leq b \leq 13$)和 c ($0 \leq c < 300$)，分别代表团队ID、问题ID和提交时间（以分钟为单位）。保证提交次数不减。

输出

对于每次提交，如果应奖励与该问题对应的气球，请输出问题 ID
否则输出 0。

例子

standard input	standard output
8	1
1 1 10	2
1 2 20	3
1 3 30	0
1 4 250	0
1 5 260	0
1 6 270	0
1 7 280	1
2 1 290	

此页故意留空

问题 J. 克洛诺斯的回响

3142 年，在泰坦环的深处，人类崩溃前的最后一份记忆档案被封存在计时金库中。它的入口由回声表盘守卫，这是一个量子纠缠锁，由排列在一根细丝中的 n 时间环组成。每个环显示 0 到 $m - 1$ 范围内的相位值，表明其与宇宙节奏一致。最初，第 i 个环显示值 a_i 。

Echo Dial 只能通过共振调谐进行调整，并遵循以下规则：

- 在单个操作中，您可以选择任何相位值 $x \in [0, m)$ 并同时调整当前显示 x 的所有环：
 - 向前调整： $x \rightarrow (x + 1) \bmod m$ ，或
 - 向后调整： $x \rightarrow (x - 1) \bmod m$ 。
- 此操作会影响整个表盘中当前在相位 x 共振的每个环，无论位置如何。

回声神谕，避难所的感知守护者，发布了 q 诊断试验。每个试验都以查询 (l, r, v) 的形式给出，并要求将位置 l 到 r （包括）中的所有环设置为相位 v 所需的最小谐振调谐操作次数。

所有试验都是假设的。对于每个查询，Echo Dial 始终从其原始初始状态开始，并且任何尝试都不会改变拨号的实际配置。作为最后一位量子档案管理员，您的职责是有效地回答所有 q 查询。

输入

第一行包含三个整数 n 、 q ($1 \leq n, q \leq 2 \times 10^5$) 和 m ($1 \leq m \leq 10^9$)。
第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i < m$)，表示每个环的初始相位。
接下来的 q 行包含三个整数 l 、 r ($1 \leq l \leq r \leq n$) 和 v ($0 \leq v < m$)，描述一个诊断试验。

输出

对于每个查询，输出一行包含一个整数，表示将位置 $[l, r]$ 中的所有环设置为阶段 v 所需的最小操作数，假设表盘从其原始状态开始并在查询之间保持不变。

例子

standard input	standard output
4 6 7	5
2 0 2 5	4
1 4 3	2
2 4 6	4
1 3 1	4
1 3 5	0
3 4 0	
3 3 2	

此页故意留空

Problem K. Relay Jump

There are n frogs numbered 1 through n , each initially located at an integer lattice point in the Cartesian plane. An external disturbance suddenly stimulates one of the frogs, triggering a chain reaction in which the frogs act according to the rules below.

When frog i ($1 \leq i \leq n$) is *stimulated*, it can choose to either:

- jump toward some other frog j ($1 \leq j \leq n, j \neq i$) and instantly land at the reflection of its current position across the point currently occupied by frog j (if frog i coincides with frog j before the jump, then it lands at the coinciding point); immediately after this jump, frog j becomes the next and only frog to be stimulated;
- or stay where it is, in which case all jumps end and no frog remains stimulated.

A frog may be stimulated multiple times at different moments, while some frogs may never be stimulated. You observe that the first stimulated frog is numbered s , but there are too many jumps for you to capture the entire process. Fortunately, you recognize every frog, so you record both their initial positions $P_1, P_2, \dots, P_n$ and their final positions $Q_1, Q_2, \dots, Q_n$ after all jumps have ended. You also notice that all initial positions are **pairwise distinct**, and the final positions are **pairwise distinct** as well. Let t be the index of the last stimulated frog, i.e., the frog that chooses to stay still when stimulated, after which no more jumps occur. Please find t .

Input

The first line contains two integers n ($2 \leq n \leq 10^5$) and s ($1 \leq s \leq n$), denoting the number of frogs and the index of the first stimulated frog, respectively.

The i -th of the next n lines contains four integers px_i, py_i, qx_i , and qy_i ($-10^9 \leq px_i, py_i, qx_i, qy_i \leq 10^9$), where $P_i = (px_i, py_i)$ and $Q_i = (qx_i, qy_i)$ represent the initial and final positions of frog i in the plane.

It is guaranteed that all initial positions $P_1, P_2, \dots, P_n$ are pairwise distinct. All final positions $Q_1, Q_2, \dots, Q_n$ are pairwise distinct as well, and correspond to some sequence of at most 2×10^5 stimulations following the described rules.

Output

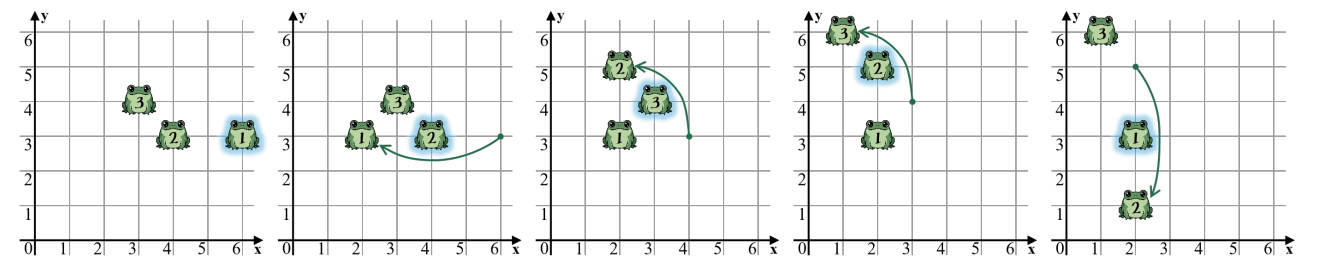
Output a single integer t , denoting the index of the frog that was last stimulated.

Example

standard input	standard output
3 1 6 3 2 3 4 3 2 1 3 4 1 6	1

Note

In the sample case, the corresponding sequence of stimulations is $[1, 2, 3, 2, 1]$.



This page is intentionally left blank

Problem L. Leo

In a future where digital logic has evolved beyond binary, a new system of signal processing, known as tri-color logic, has become the standard. This system defines four fundamental states in two categories:

- **Colored:** There are three colored states, denoted by R (Red), G (Green), and B (Blue), respectively. Each serves as a distinguishable “on” state, analogous to the traditional logic 1 signal.
- **Colorless:** The only colorless state is denoted by $*$ (Transparent). It serves as an “off” state, analogous to the traditional logic 0 signal.

Sulfox the fennec fox is designing a computing device based on tri-color logic technology, named Leo. It employs a simple combinational logic architecture, so we can regard it as a directed acyclic graph in which each node holds a state from the set $\{R, G, B, *\}$. After fixing the machine’s input scale to n , the graph is constructed as follows:

- Nodes $1, 2, \dots, n$ are *input nodes*, whose states should be given as the input to the machine. Input nodes have no predecessors.
- Nodes $n + 1, n + 2, \dots, n + m$ are *internal nodes*, whose states are computed based on the states of two predecessors, where the two predecessors of each internal node may be the same and are chosen from nodes with indices strictly less than its own. To ensure the machine’s operating speed, m (the number of internal nodes) **must not exceed** $6n$.
- The internal node $n + m$ is also the only *output node*, whose state is the output of the machine.

Every internal node is of exactly one of the following two types, depending on how it combines the signals from its two predecessors:

- **Tri-Color AND:** If the states of its predecessors are identical, its own state will be that same state. If they differ, its own state will be $*$.
- **Tri-Color OR:** If the states of its predecessors are identical, its own state will be that same state. If one predecessor’s state is $*$ while the other’s is colored, its own state will be the colored one. If they differ and both are colored, its own state will be the colored state that is different from both.

With everything prepared, we now specify Leo’s intended functionality. The states given to the n input nodes will be guaranteed to contain at least one occurrence of each of R, G, and B, where $*$ may appear any number (possibly zero) of times. For every such input, the state of the output node $n + m$ must be identical to **the third distinct colored state encountered** when examining the input nodes from node 1 to node n (ignoring all $*$). In other words, among R, G, and B it must output the colored state whose first occurrence has the largest index.

Given the machine’s fixed input scale n , please complete the design of the internal nodes, i.e., specify the number of internal nodes and the type and predecessors of each internal node.

To verify the correctness of your design, for each test, the judge will generate $\left\lfloor \frac{10^7}{n} \right\rfloor$ valid input cases, each of which will be provided as your machine’s input in turn. Your solution passes a test if your machine’s output always meets Leo’s intended functionality across all input cases.

Input

The only line contains an integer n ($3 \leq n \leq 10^5$), denoting the number of input nodes.

Output

In the first line, output an integer m ($1 \leq m \leq 6n$), denoting the number of internal nodes in your design.

Then output m lines, the i -th line containing a character t_i ($t_i \in \{ \text{'\&'}, \text{'|'} \}$) and two integers u_i and v_i ($1 \leq u_i, v_i < n + i$), denoting the type of the internal node $n + i$ and the indices of its two predecessors, where $\text{'\&'}$ represents the tri-color AND type and '|' represents the tri-color OR type.

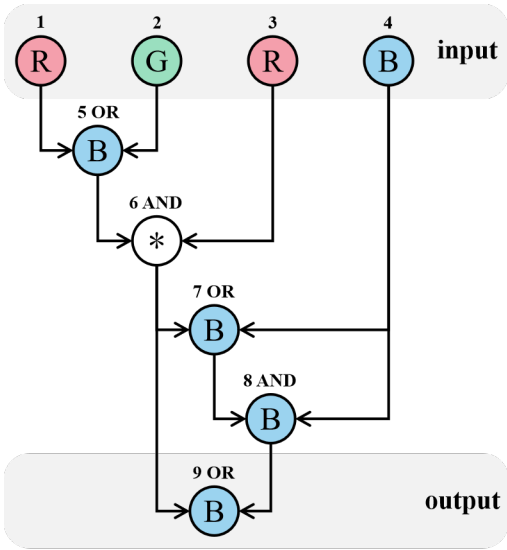
If there are multiple designs, you may output any of them. Note that you do not have to minimize m .

Example

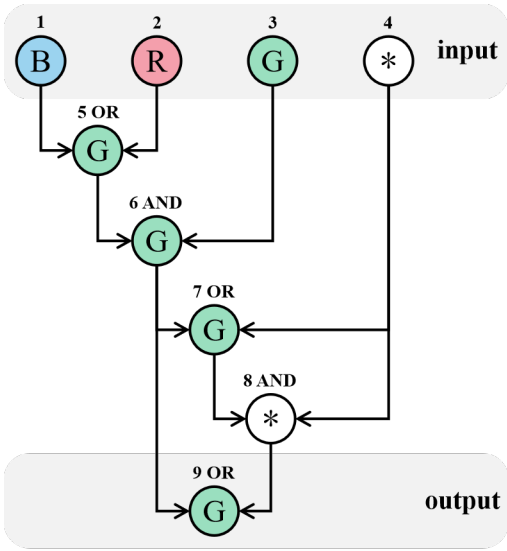
standard input	standard output
4	5 1 2 & 3 5 4 6 & 4 7 6 8

Note

It can be proven that the design in the sample case can always find the third encountered distinct colored state for every valid input of $n = 4$. For example, the figure below illustrates the machine’s computation for the input “RGRB”:



The figure below illustrates the machine’s computation for the input “BRG*”:



Problem M. The End?

At the 2025 League of Legends Season 15 World Championship, T1 battled KT Rolster in a full five-game series, ultimately defeating KT with a 3-2 score to claim the championship. This match was exceptionally significant, giving birth to multiple new historical records.

As T1's star mid-laner, Faker has now achieved six World Championship titles in his personal career with this victory, further cementing his status as the "Greatest Player of All Time" in League of Legends. Besides Faker, his teammates Oner, Gumayusi, and Keria also accomplished the remarkable feat of securing three consecutive World Championship titles. Meanwhile, the newly joined top-laner Doran filled the gap in his personal accolades by winning his first World Championship.

The journey for the LPL region, however, was full of drama. The highly-touted first seed Bilibili Gaming (BLG) suffered a surprising upset loss to the North American team 100 Thieves in the very first round of the Swiss Stage. They were then eliminated in the crucial qualification match by Top Esports (TES), stopping at the Top 16 as a first seed and becoming one of the major upsets of the tournament. TES itself was swept 3-0 by T1 in the semi-finals, leaving the LPL region absent from the finals. Has the LPL reached its conclusion?

When we look back at the journey of the two finalist teams, everything began with the life-or-death knockout stage. The eight-team single-elimination bracket stipulated that teams were placed into the bracket based on their performance in the Swiss Stage, entering a series of head-to-head battles where the loser was immediately eliminated. The entire stage consisted of three rounds: the quarter-finals, where eight teams were narrowed down to four; the semi-finals, where these four teams produced two finalists; and the finals, where one ultimate champion was crowned from these two teams.

For Season 16, can our Team 1 win the championship? Please find the maximum probability of Team 1 winning the entire tournament.

More specifically, there are 8 teams participating in a single-elimination tournament. Each team i has two strength values a_i and b_i .

Before the tournament starts, you need to assign each team to a distinct seed from 1 to 8. The tournament follows the standard single-elimination bracket format:

- Round 1: seed 1 vs seed 2, seed 3 vs seed 4, seed 5 vs seed 6, seed 7 vs seed 8
- Round 2: winner of (1, 2) vs winner of (3, 4), winner of (5, 6) vs winner of (7, 8)
- Round 3 (Final): winner of upper bracket (1, 2, 3, 4) vs winner of lower bracket (5, 6, 7, 8)

When two teams compete against each other, the team with the smaller seed number uses its a -value as its strength, and the team with the larger seed number uses its b -value as its strength. If a team with strength x competes against a team with strength y , the probability that the first team wins is $\frac{x}{x+y}$.

You need to find the maximum probability of team 1 winning the entire tournament, considering all possible assignments of teams to seeds.

Input

The input consists of 8 lines. The i -th line contains two integers a_i and b_i ($1 \leq a_i, b_i \leq 100$), denoting the two strength values of team i .

Output

Output a line containing a real number, denoting the maximum probability that team 1 wins the tournament.

Your answer is acceptable if its absolute or relative error does not exceed 10^{-6} . Formally speaking, suppose that your output is a and the jury's answer is b , and your output is accepted if and only if $\frac{|a-b|}{\max(1, |b|)} \leq 10^{-6}$.

Examples

standard input	standard output
10 80 20 70 30 60 40 50 50 40 60 30 70 20 80 10	0.329505822460368
100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100	0.1250000000000000